FreeGAD: A Training-Free yet Effective Approach for Graph Anomaly Detection

Yunfeng Zhao, Yixin Liu, Shiyuan Li, Qingfeng Chen, Yu Zheng, Shirui Pan
Chloe Kim
March 2, 2026
CIKM ‘25

Motivation

- Deep learning-based GAD requires costly training
  - Hundreds of epochs needed; training dominates deployment time
  - GNN backpropagation over entire graph is memory-intensive
  - Most baselines fail on T-Finance (39k nodes) with a 24GB GPU
- Poor scalability limits real-world deployment
  - Time-sensitive systems cannot afford long training cycles
- Key observation: training may not be as critical as assumed
  - Training-free CoLA and TAM variants drop only ~5% on average

Question

- Is it truly necessary to heavily train GNN-based GAD models to achieve effective anomaly detection?
- Can we design a GAD approach that achieves competitive performance without the need for training?

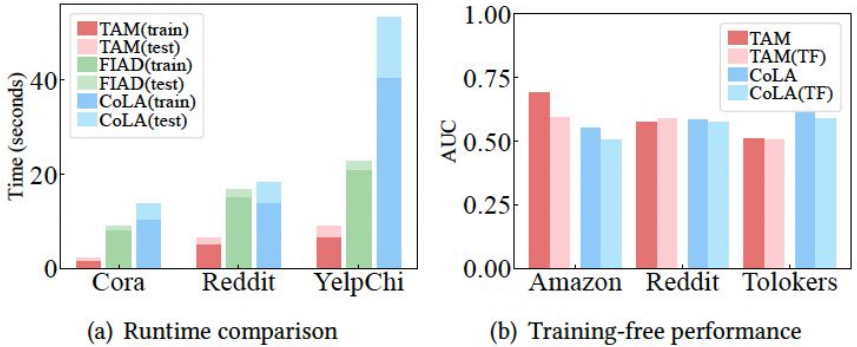


Figure 1: (a): The efficiency of different GAD methods, with the runtime square-rooted for comparison. (b): The difference of GAD methods with and without training.

Existing GAD Approaches

- Reconstruction-based (DOMINANT, AnomalyDAE)
  - Graph autoencoders using reconstruction error as anomaly signal
  - Require full gradient-descent training
- Contrastive learning-based (CoLA, ANEMONE, PREM)
  - GNN frameworks with contrastive objectives
  - Multiple estimation rounds — computationally expensive
- Affinity-based (TAM)
  - Truncated affinity maximization for one-class homophily modeling
- Common limitation: all deep GAD methods require iterative optimization

FreeGAD Framework

1. Affinity-Gated Residual Encoder
   - generate anomaly sensitive node representations
2. Anchor Node Selection
   - choose the pseudo normal/abnormal nodes that characterize the underlying data distribution
3. Anchor-Guided Anomaly Scoring
   - measure node-level abnormality by statistically evaluating the distance between the representations of each node and the selected anchor nodes

FreeGAD Framework

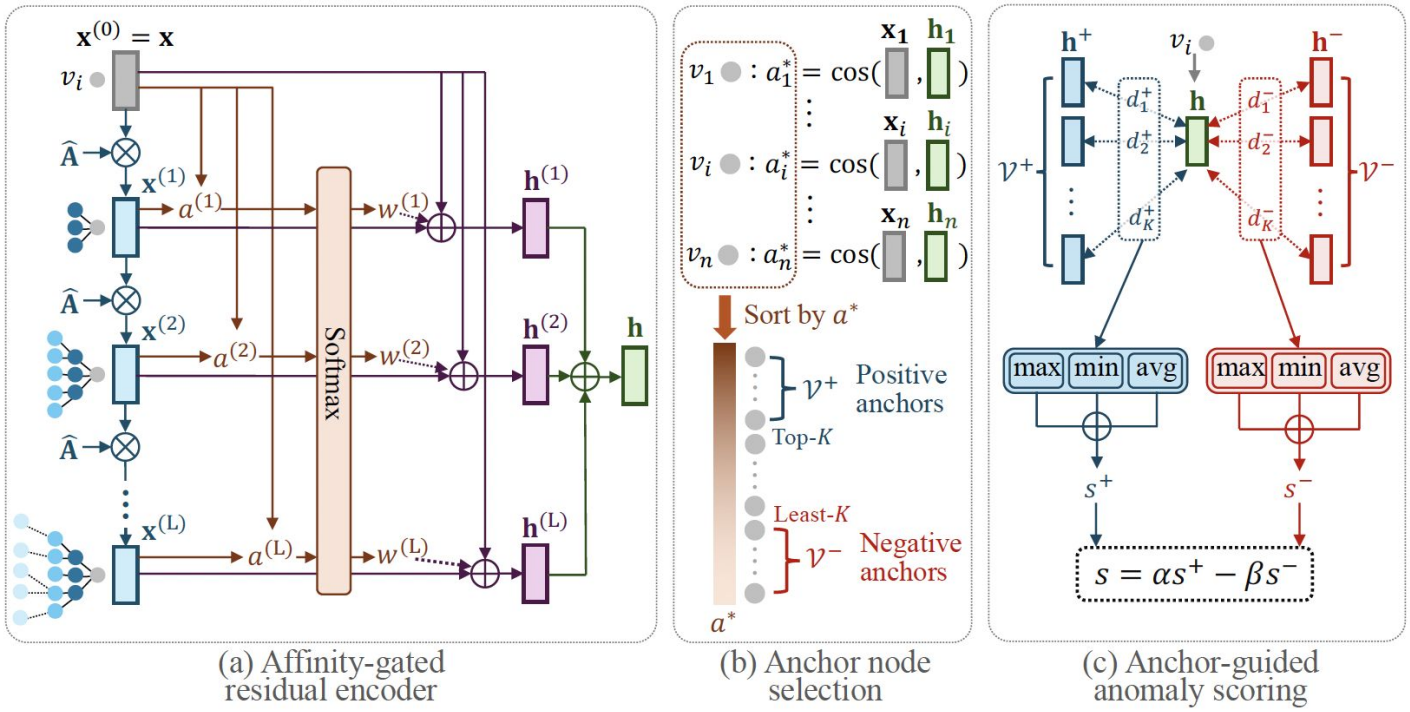
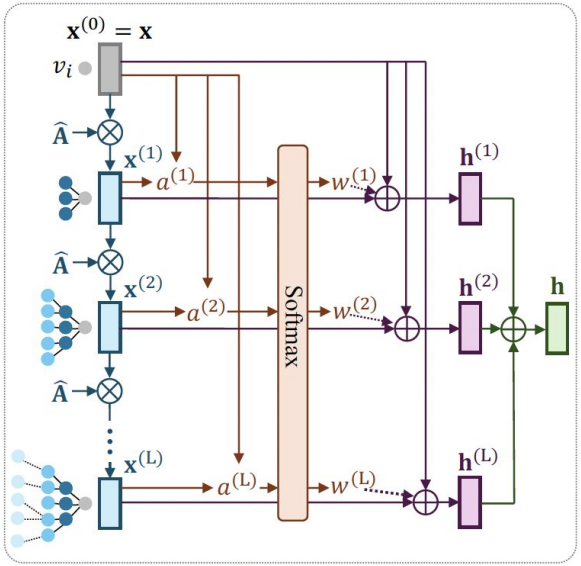


Figure 2: The overall pipeline of FreeGAD. In blocks (a) and (c), we omit the subscript "$_i$" for variables x, a, w, h, d, and s for simplicity.

Affinity-Gated Residual Encoder

1. **Multi-Hop Propagation (No over-smoothing and model degradation)**
   a. Spread features across neighbors for L hops
   b. No feature transformation operation (propagation only)
2. **Affinity (Captures anomaly-sensitive knowledge)**
   a. “Node affinity is a key property that strongly correlates with node abnormality”
   b. Compare original vs propagated features
   c. High similarity -> normal
   d. Low similarity -> potential anomaly
   e. Consider multi-order affinity
      i. For real-world high-order affinity



(a) Affinity-gated residual encoder

Affinity-Gated Residual Encoder Cont.

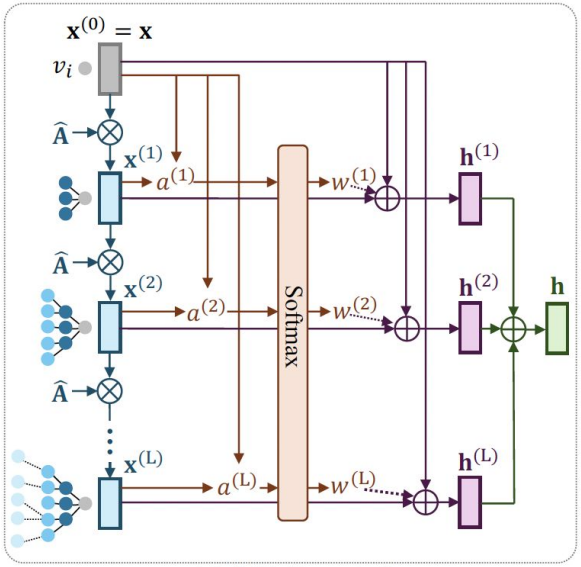
3. Affinity-Gated Residual

a. Residual = add original feature back in
   i. Preserve ego information within the features
b. High affinity -> keep more original features
c. Low affinity -> emphasize neighbor-influenced features
d. Avoids over-smoothing caused by multi-hop propagations

4. Multi-Hop Aggregation

e. Combine all hops to capture local + global context

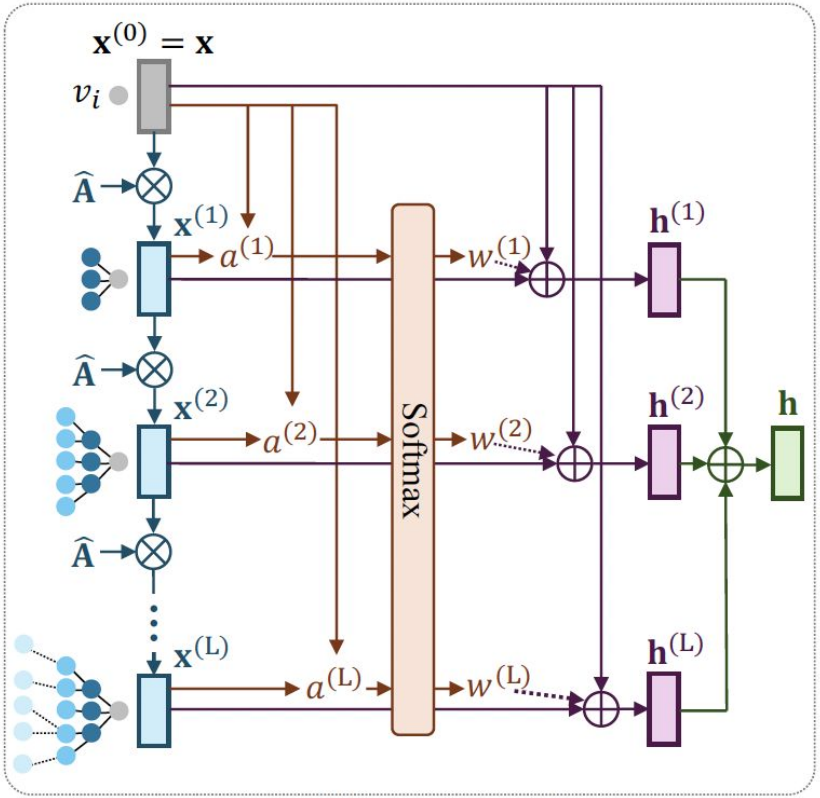
- Multi-scale aare, affinity-aware, training-free, and original space-preserving



(a) Affinity-gated residual encoder

Affinity-Gated Residual Encoder

- Multi-Hop Propagation
  - $\mathbf{X}^{(l)} = \hat{\mathbf{A}}\mathbf{X}^{(l-1)}$
- Affinity Estimation
  - $a_i^{(l)} = \dfrac{\sum_{j=1}^{m} \mathbf{x}_{ij}^{(0)} \cdot \mathbf{x}_{ij}^{(l)}}{\|\mathbf{x}_i^{(0)}\|^2 \cdot \|\mathbf{x}_i^{(l)}\|^2 + \sigma},$
  - $w_i^{(l)} = \dfrac{\exp\left(a_i^{(l)}\right)}{\sum_{k=1}^{L} \exp\left(a_i^{(k)}\right)}.$
- Affinity-Gated Residual
  - $\mathbf{h}_i^{(l)} = (1 - w_i^{(l)})\mathbf{x}_i^{(l)} + w_i^{(l)}\mathbf{x}_i^{(0)}.$
- Multi-Hop Mixing
  - $\mathbf{h}_i = \dfrac{1}{L}\sum_{l=1}^{L} \mathbf{h}_i^{(l)},$



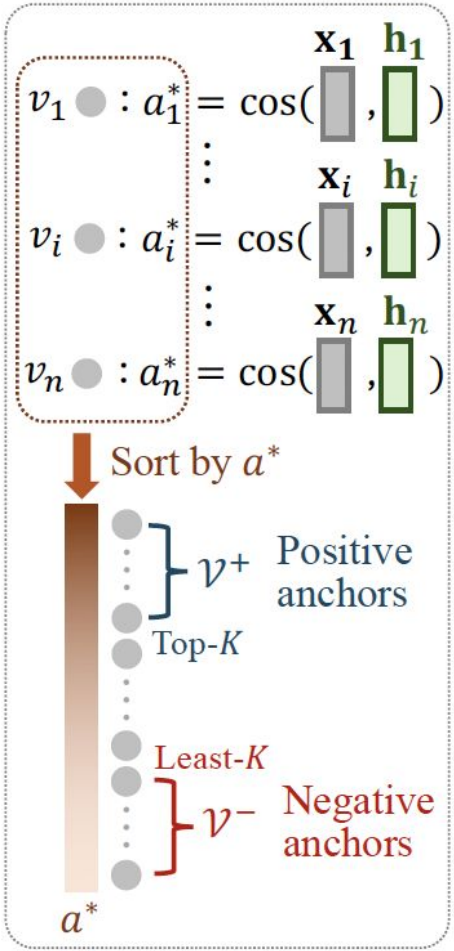
(a) Affinity-gated residual encoder

Anchor Node Selection

- Affinity-based pseudo-labeling

- Compute overall affinity between original and final representation
- Nodes that strongly agree with their neighborhood -> high affinity

- Top-K highest affinity -> pseudo-normal anchors
- Bottom-K lowest affinity -> pseudo-anomalous anchors



(b) Anchor node selection

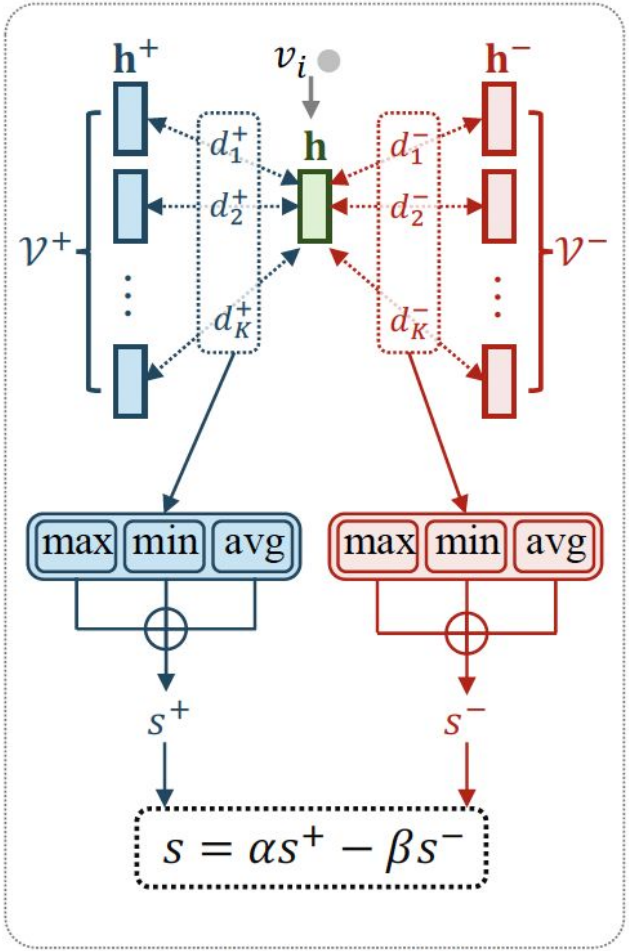
Anchor-Guided Anomaly Scoring

- Compute distances from each node to:
  - Positive (pseudo-normal) anchors
  - Negative (pseudo-anomalous) anchors

- Summarize “how far” using simple statistics
- Use min + max + average distance to get a robust score

- Final anomaly score balances both signals
- Far from normal anchors -> more anomalous
- Close to anomalous anchors -> more anomalous

- Weighted combination with α and β controls the trade-off



(c) Anchor-guided anomaly scoring

FreeGAD Framework

- Affinity-Gated Residual Encoder
  - Multi-hop propagation without feature transformation
  - Affinity between raw and propagated features used as gating scalar
  - Residual design mitigates over-smoothing; representations averaged across layers
- Anchor Node Selection
  - Top-K nodes by affinity: positive anchors (pseudo-normal)
  - Least-K nodes by affinity: negative anchors (pseudo-anomalous)
- Anchor-Guided Anomaly Scoring
  - Euclidean distances to positive and negative anchor sets
  - s = alpha * s+ - beta * s- where s+/s- combine min, max, avg of distances
- Entire process runs in a single forward pass, no optimization loop

FreeGAD Framework

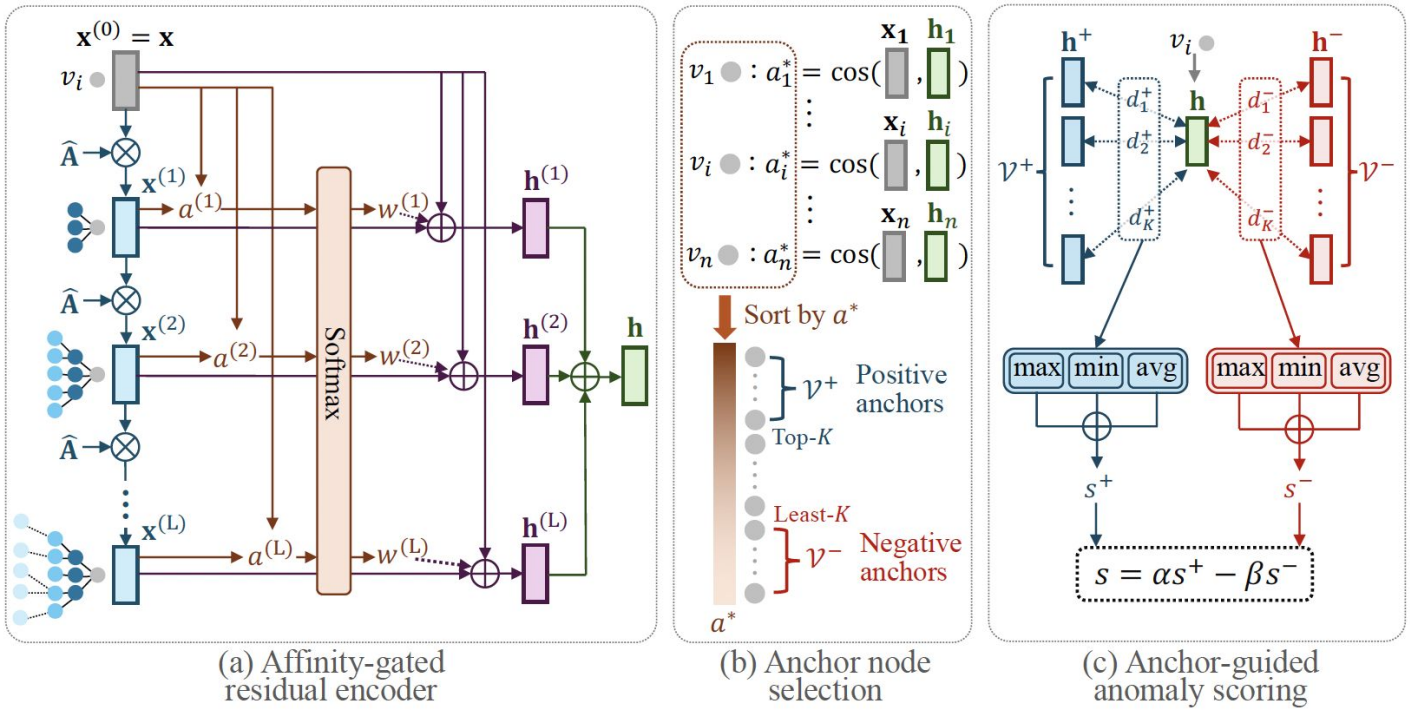


Figure 2: The overall pipeline of FreeGAD. In blocks (a) and (c), we omit the subscript "$_i$" for variables x, a, w, h, d, and s for simplicity.

Experimental Setup

- 10 benchmark datasets across diverse domains
  - Real anomalies (7): Amazon, Reddit, Tolokers, YelpChi, T-Finance, Questions, Elliptic
  - Injected anomalies (3): Cora, BlogCatalog, Flickr
  - Scale: 2.7k to 203k nodes; up to 21M+ edges
- 10 baselines compared
  - Shallow: Radar, ANOMALOUS
  - Deep: DOMINANT, AnomalyDAE, CoLA, CONAD, PREM, TAM, GADAM, FIAD
- Metrics: AUROC and AUPRC (mean over 5 trials)
- FreeGAD hyperparameters: alpha, beta in [0,1]; L in [1,20]; K in [10,100]
- Hardware: Intel i5-13600K CPU, NVIDIA RTX 3090 GPU (24GB)

Table 1: Statistics of the datasets.

Name	#Nodes	#Edges	#Dim.	%Ano.
Dataset with real anomalies				
Amazon	10,244	175,608	25	6.76
Reddit	10,984	168,016	64	3.33
Tolokers	11,758	519,000	10	21.8
YelpChi	23,831	49,315	32	5.10
T-Finance	39,357	21,222,543	10	4.60
Questions	48,921	153,540	301	2.98
Elliptic	203,769	234,355	166	2.23
Datasets with injected anomalies				
Cora	2,708	5,429	1,433	5.53
BlogCatalog	5,196	171,743	8,189	5.77
Flickr	7,575	239,738	12,047	5.94

Performance Comparison

- FreeGAD achieves SOTA across 7 real-world datasets
  - Amazon: 88.57 AUROC, next best 68.87 (TAM)
  - T-Finance: 92.13, most baselines OOM; only 3 others ran
  - YelpChi: 78.55; Tolokers: 67.35; Questions: 64.27; Elliptic: 77.25
- Competitive on injected anomaly datasets
  - Outperforms all shallow methods; comparable to leading deep methods
- Stable, deterministic results. No variance from random initialization
- Scalability: runs on graphs where most baselines fail due to out-of-memory issues

Table 2: Anomaly detection performance in terms of AUROC and AUPRC (in percent, mean±std). The best results are highlighted in bold and underlined. OOM: out-of-memory (24GB GPU). OOT: out-of-time (12 hours). We do not report the standard deviation of FreeGAD since it is a training-free method, and its results are deterministic, without any randomness introduced by training processes or initialization.

Metric	Method	Amazon	Reddit	Tolokers	YelpChi	T-Finance	Questions	Elliptic	Cora	BlogCatalog	Flickr
AUROC	Radar	55.43±0.00	51.22±0.00	51.71±0.00	50.61±0.00	OOM	OOM	OOM	69.32±0.00	61.16±0.00	59.52±0.00
	ANOMALOUS	55.46±0.00	55.95±4.78	46.48±0.28	48.77±1.04	OOM	OOM	OOM	69.38±0.00	70.86±0.01	59.52±0.00
	DOMINANT	49.56±0.65	55.70±0.77	54.43±0.11	41.87±1.87	OOM	OOM	OOM	88.92±0.14	76.34±0.29	77.19±1.59
	AnomalyDAE	51.60±3.31	55.18±0.43	52.20±2.94	59.45±2.32	OOT	OOT	OOT	80.66±0.25	75.61±0.02	74.87±0.09
	CoLA	55.23±0.94	**58.24**±0.44	61.17±0.37	36.46±0.58	23.45±0.19	46.37±0.37	OOM	89.59±0.31	78.30±0.21	70.37±0.39
	CONAD	51.02±0.23	54.11±0.59	49.47±2.36	56.65±1.07	OOM	OOM	OOM	71.82±0.02	72.54±0.02	72.82±0.02
	TAM	68.87±0.72	57.17±0.13	50.88±0.00	OOM	OOM	OOM	OOM	92.35±0.23	75.58±0.20	73.26±0.45
	PREM	62.61±11.85	51.87±6.15	53.48±4.04	73.93±1.51	75.30±12.31	57.16±1.29	71.23±11.08	**95.11**±0.23	75.06±0.74	**86.53**±0.37
	GADAM	67.09±3.12	58.18±0.38	55.42±0.56	65.53±1.66	80.20±7.85	43.70±0.37	56.42±0.35	93.93±0.25	**81.58**±0.18	72.26±0.21
	FIAD	52.36±8.07	55.72±1.03	49.73±1.98	43.27±3.19	OOM	OOM	OOM	87.43±1.31	75.47±2.02	77.01±2.01
	FreeGAD	**88.57**	57.21	**67.35**	**78.55**	**92.13**	**64.27**	**77.25**	84.85	74.84	74.73
AUPRC	Radar	7.30±0.00	3.53±0.00	23.24±0.00	5.88±0.00	OOM	OOM	OOM	11.86±0.00	14.80±0.00	19.88±0.00
	ANOMALOUS	7.30±0.00	3.86±0.56	20.14±0.00	5.51±0.15	OOM	OOM	OOM	25.23±0.00	30.62±0.00	19.88±0.00
	DOMINANT	6.08±0.08	3.71±0.06	27.95±0.03	4.24±0.34	OOM	OOM	OOM	37.81±0.44	34.55±0.05	37.10±0.05
	AnomalyDAE	7.30±1.18	3.71±0.02	26.82±1.12	7.53±0.59	OOT	OOT	OOT	26.71±0.61	23.01±0.09	16.78±0.24
	CoLA	8.28±0.43	4.01±0.05	27.25±0.31	3.75±0.06	2.79±0.05	2.55±0.02	OOM	45.79±2.66	27.09±0.98	21.36±0.31
	TAM	29.73±1.28	4.32±0.03	22.98±0.00	OOM	OOM	OOM	OOM	48.24±1.54	33.88±0.77	24.70±1.87
	PREM	8.96±2.38	3.95±0.73	23.25±1.67	10.67±1.34	33.05±24.39	3.49±0.24	**7.33**±3.38	65.99±0.23	37.31±0.74	**46.37**±0.37
	GADAM	15.87±7.48	**5.01**±0.23	23.65±0.35	10.14±0.32	15.72±6.12	2.52±0.10	3.03±0.05	**71.83**±1.91	**37.36**±0.36	23.19±0.43
	CONAD	6.23±0.03	4.31±0.10	21.13±0.11	7.19±0.44	OOM	OOM	OOM	24.86±0.05	33.01±0.00	37.57±0.01
	FIAD	6.71±1.23	3.74±0.04	29.31±0.82	4.32±0.35	OOM	OOM	OOM	29.31±3.69	25.44±6.53	19.55±3.56
	FreeGAD	**75.06**	3.85	**32.17**	**15.80**	**73.71**	**7.01**	6.51	49.75	34.03	38.68

Efficiency Analysis

- Total runtime orders of magnitude lower, no training phase
  - Amazon: FreeGAD 0.032s vs CoLA 267s, AnomalyDAE 114s, TAM 47s
  - Testing time comparable to or better than most baselines

Table 3: Training and testing time of baselines and FreeGAD in seconds. The shortest overall times are highlighted in bold and underlined.

Method	Amazon		Reddit		Tolokers		YelpChi		Cora		BlogCatalog		Flickr	
	train	test	train	test	train	test	train	test	train	test	train	test	train	test
Radar	2.0061	0.0204	1.8485	0.0213	1.9994	0.0229	5.8551	0.0865	0.4789	0.008	1.4281	0.1333	1.8316	0.4085
ANOMALOUS	2.1543	0.0135	2.3067	0.0101	2.2285	0.0092	7.0728	0.027	1.252	0.0102	23	0.1734	70	0.5362
DOMINANT	2.3293	0.1633	2.6146	0.1886	2.8165	0.2138	11	0.8201	0.5937	0.0409	4.5385	0.4315	12	1.2824
AnomalyDAE	113	1.1018	118	1.2149	134	1.4219	345	3.4737	32	0.3022	101	0.923	139	1.3636
CoLA	247	20	196	18	172	15	1642	167	111	10	161	15	236	22
CONAD	12	1.3023	13	1.3489	13	1.561	39	4.4816	1.85	0.586	14	1.0551	20	1.7071
TAM	46	1.2557	27	1.4499	73	1.5347	44	5.4196	3.0339	0.2301	42	0.4958	59	0.8114
PREM	0.4613	0.0066	0.3637	0.0075	0.3471	0.0065	0.508	0.0108	0.3056	0.0042	0.9174	0.0054	1.8403	0.0075
GADAM	1.0099	0.0102	0.8122	0.0098	0.8889	0.0124	0.738	0.0213	0.7409	0.0054	1.6262	0.007	2.199	0.0085
FIAD	227	2.0116	233	2.1419	242	2.1964	442	3.4145	67	0.497	215	1.9857	257	2.0871
FreeGAD	**0.0317**		**0.0338**		**0.0141**		**0.0357**		**0.0601**		**0.1641**		**0.3142**	

Efficiency Analysis

- Significantly lower GPU memory consumption
  - Amazon: FreeGAD 32 MB vs DOMINANT 2850 MB, TAM 3968 MB, FIAD 4085 MB
  - Closest methods (PREM, GADAM) still consume several times more memory
- Higher memory on high-dimensional datasets (Cora, BlogCatalog, Flickr)
  - Operates on raw features without dimensionality reduction
  - Still outperforms most baselines on these datasets
- Ideal for time-sensitive and resource-constrained deployments

Efficiency Analysis

Table 4: Maximum GPU memory usage (in MB) during training and testing for baselines and FreeGAD

Method	Amazon		Reddit		Tolokers		YelpChi		Cora		BlogCatalog		Flickr	
	train	test	train	test	train	test	train	test	train	test	train	test	train	test
Radar	2814	2416	3257	2796	3715	3187	15202	13036	316	288	2248	1770	4797	3766
ANOMALOUS	1223	1223	1426	1426	1605	1605	6548	6548	263	263	2426	2111	5185	4502
DOMINANT	2850	2035	3290	2351	3759	2682	15284	10910	272	228	1448	1225	3046	2567
AnomalyDAE	3364	3335	3865	3844	4441	4403	17978	17945	343	338	1879	1847	3949	3903
CoLA	423	404	487	467	552	532	2193	2173	79	63	432	376	922	729
CONAD	2452	1639	2815	1876	3246	2173	13076	8706	253	167	1677	1185	3538	2488
TAM	3968	3842	4536	4351	5150	4899	19742	17590	440	551	1535	1620	3036	3056
PREM	91	34	109	43	114	49	219	84	111	59	1191	555	2510	1149
GADAM	96	35	98	33	109	40	153	38	108	51	746	371	1515	756
FIAD	4085	3281	4710	3784	5413	4351	21817	17471	450	364	2605	2073	5498	4361
FreeGAD	32		30		74		30		112		1144		2445	

Ablation Study

- w/o Multi-hop: representations at the last layer serve as the final representations
  - Large performance drop across all datasets
  - Confirms importance of capturing high-order neighbourhood information
- w/o Anchor (random selection): anchor nodes are randomly selected rather than affinity-based
  - Severe drop. e.g. Cora 84.85 to 28.62 AUROC
  - Affinity-based anchor selection is critical for distinguishing anomalies
- w/o Negative or w/o Positive: only positive or negative score contributes to final anomaly score
  - Noticeable declines
  - Both scores are complementary; removing either hurts performance
- Single-statistic scoring suboptimal
  - Avg best for citation/review networks; Min for social; Max+Min for fraud detection
  - Combining all three consistently outperforms any single metric

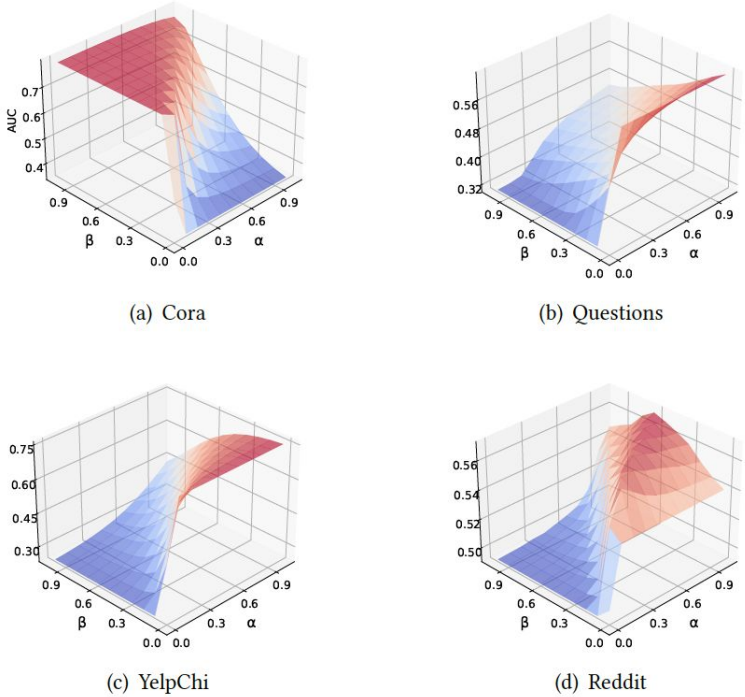
Ablation Study

Table 5: Performance of FreeGAD and its variants in terms of AUROC (in percent). The best results are highlighted in bold and underlined.

Variant	Cora	Reddit	YelpChi	Elliptic
FreeGAD	**84.85**	**57.21**	**78.55**	**77.25**
w/o Multi-hop	66.80	48.62	54.85	56.21
w/o Anchor	28.62	52.70	75.16	52.13
w/o Negative	30.83	54.05	78.55	52.84
w/o Positive	84.83	48.24	25.58	65.61
w/ Max Score	70.21	54.03	73.85	76.48
w/ Min Score	77.34	56.78	75.34	76.34
w/ Avg Score	84.68	55.99	77.25	41.37

Parameter Analysis

- Trade-off weights alpha and beta
  - Dataset-dependent: Cora (citation) requires higher beta (negative scores)
  - Social and co-review networks require higher alpha (positive scores)
  - Optimal balance varies with domain-specific anomaly patterns



(a) Cora (b) Questions (c) YelpChi (d) Reddit

Figure 3: The sensitivity of FreeGAD in terms of α and β.

Parameter Analysis Cont.

- Propagation layer L
  - Larger L (>= 8) consistently preferred across all datasets
  - Multi-hop neighbourhood capture is crucial for effective anomaly detection
  - Suggests shallow GNNs in existing methods may lead to suboptimal performance

- Number of anchor nodes K

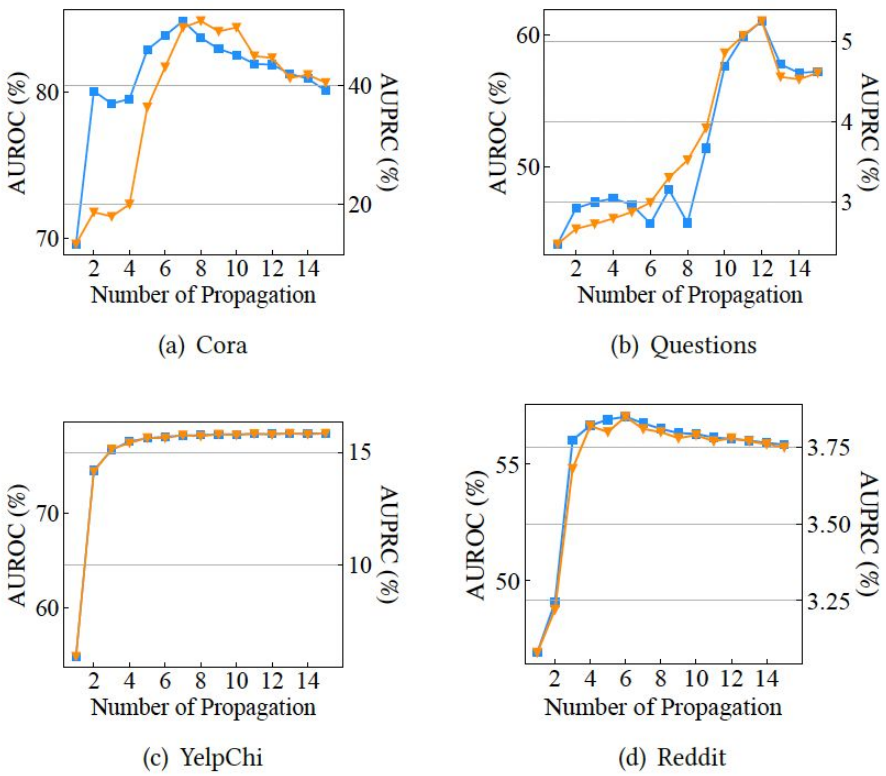


Figure 4: The sensitivity of FreeGAD on layer number L.

Key Contributions

- Training-free GAD methodology
  - First to demonstrate competitive GAD without any training or optimization
  - Eliminate need for resource-intensive training process
  - Challenges prevailing assumption that training is critical in deep GAD
- Affinity-Gated Residual Encoder, Anchor Node Selection, Anchor-Guided Anomaly Scoring
- Strong empirical results
  - SOTA on 6 out of 10 datasets; runs where most baselines fail

Limitations & Future Works

Limitations

- Design choices are mainly empirically driven
- Assumes neighbourhood consistency (may struggle in heterophilic or camouflaged settings)
- Requires manual tuning of α and β and other hyperparameters

Future Work

- Theoretical analysis under different graph structures
- Automatic assignment of α and β and other hyperparameters
    - Adaptive anchor selection

Conclusion

- Proposed FreeGAD: a training-free, effective graph anomaly detection method
- Key insight: model training contributes less to GAD than previously assumed
- FreeGAD operates in a single forward pass with zero learnable parameters
- Extensive experiments across 10 datasets demonstrate:
  - Superior or competitive anomaly detection performance
  - Extremely low deployment time
  - Excellent scalability to large and dense graphs
- Highlights the promise of training-free approaches for advancing GAD research